# Invoice-CRM

## Implementation Plan — Audit Remediation

**For:** Developer picking up audit work **Target:** Before Supabase project migration

CRITICAL — Read First: No RLS is enabled on any table. The Supabase anon key (visible in every browser bundle) can currently read and write everything, including Stripe secret keys stored in payment_gateways. Treat those Stripe secrets as already leaked. Execute phases in order: Phase 0 → 1 → 2 → 3. Do not reorder — each phase unblocks the next, and enabling RLS before the code is ready will break the public invoice page.

## Phase Overview

| Phase | What | Duration |
|---|---|---|
| 0 | Prep (rotate keys, branches, backups) | 1-2 h |
| 1 | Critical security fixes (deploy-blocking) | 1-2 days |
| 2 | API-route auth hardening | 1 day |
| 3 | RLS rollout (staging -> prod) | 1 day |
| 4 | Schema fixes | 1 day |
| 5 | Cleanup (Low findings) | ongoing |

**Everything in Phases 0-3 must ship before the Supabase project migration. Phases 4-5 can follow.**

## Phase 0 — Prep

Estimated duration: 1-2 hours

### 0.1 Create a working branch

```
git checkout main
git pull
git checkout -b security/audit-remediation
```

### 0.2 Rotate Stripe keys (do this NOW, before anything else)

The anon key has had read access to payment_gateways.live_secret_key. Assume it is compromised.

- Stripe Dashboard > Developers > API keys > Roll key on the live secret key.
- Stripe Dashboard > Developers > Webhooks > your endpoint > Roll secret.
- Update the payment_gateways rows in Supabase with the new keys via the SQL editor:

```
update public.payment_gateways
set live_secret_key = 'sk_live_NEW_KEY_HERE',
    live_publishable_key = 'pk_live_NEW_KEY_HERE',
    updated_at = now()
where id = <gateway id>;
```

- Update STRIPE_WEBHOOK_SECRET in every .env and deployment target (local, preview, prod on Vercel).

## 0.3 Back up the database

```
pg_dump "postgresql://postgres:<PW>@db.<ref>.supabase.co:5432/postgres"
  --no-owner --no-privileges
  -f d:/bmybrand-invoice/backup/pre-rls-$(date +%Y%m%d).sql
```

If pg_dump is not installed, get it from postgresql.org/download/windows (Command Line Tools component only).

## 0.4 Audit environment variables

Confirm every target (local, preview, production) has these set. Never use defaults or empty strings in production.

| Variable | Purpose | Where |
|---|---|---|
| `NEXT_PUBLIC_SUPABASE_URL` | Project URL | client + server |
| `NEXT_PUBLIC_SUPABASE_ANON_KEY` | Anon key | client + server |
| `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_DEFAULT_KEY` | Anon key (dupe name used by some code) | server |
| `SUPABASE_SERVICE_ROLE_KEY` | Service role key | server ONLY |
| `STRIPE_SECRET_KEY` | Fallback Stripe secret | server |
| `STRIPE_WEBHOOK_SECRET` | Webhook verification | server |
| `INVOICE_LINK_SECRET_KEY` | Invoice-token HMAC | server |
| `RESEND_API_KEY` | Email sending | server |
| `VAPID_PUBLIC_KEY` / `VAPID_PRIVATE_KEY` | Web push | server |

# Phase 1 — Critical Security Fixes

Estimated duration: 1-2 days. Each item: file to change, what to change, why, and how to test.

## 1.1 Fix SSRF in proxy-image

File: `src/app/api/proxy-image/route.ts` | Audit ref: C1

Replace the whole file. Add your known-good image hosts to ALLOWED_HOSTS (Supabase storage, CDN). The DNS + HTTPS check protects you even if the allowlist is empty, but a populated allowlist is stronger.

```
import { NextResponse } from 'next/server'
import dns from 'node:dns/promises'

const ALLOWED_HOSTS = new Set<string>([
  // 'yobzpzzpxfsryysbyher.supabase.co',
  // 'cdn.yourcompany.com',
])
const MAX_BYTES = 5 * 1024 * 1024  // 5 MB
const FETCH_TIMEOUT_MS = 5000

function isPrivateIp(ip: string): boolean {
  if (/^10\./.test(ip)) return true
  if (/^127\./.test(ip)) return true
  if (/^169\.254\./.test(ip)) return true
  if (/^192\.168\./.test(ip)) return true
  if (/^172\.(1[6-9]|2\d|3[01])\./.test(ip)) return true
  if (ip === '::1' || ip.startsWith('fc') || ip.startsWith('fd')) return true
  return false
}

export async function GET(req: Request) {
  const input = new URL(req.url).searchParams.get('url') || ''
  let parsed: URL
  try { parsed = new URL(input) }
  catch { return NextResponse.json({ error: 'Invalid URL' }, { status: 400 }) }

  if (parsed.protocol !== 'https:')
    return NextResponse.json({ error: 'HTTPS only' }, { status: 400 })

  if (ALLOWED_HOSTS.size > 0 && !ALLOWED_HOSTS.has(parsed.hostname))
    return NextResponse.json({ error: 'Host not allowed' }, { status: 400 })

  const addresses = await dns.lookup(parsed.hostname, { all: true })
  if (addresses.some(a => isPrivateIp(a.address)))
    return NextResponse.json({ error: 'Private address' }, { status: 400 })

  const controller = new AbortController()
  const timer = setTimeout(() => controller.abort(), FETCH_TIMEOUT_MS)
  try {
    const res = await fetch(parsed.toString(), { signal: controller.signal })
    if (!res.ok) return NextResponse.json({ error: 'Upstream failed' }, { status: 502 })
    const ct = res.headers.get('content-type') || ''
    if (!ct.startsWith('image/'))
      return NextResponse.json({ error: 'Not an image' }, { status: 415 })
    const buf = await res.arrayBuffer()
    if (buf.byteLength > MAX_BYTES)
      return NextResponse.json({ error: 'Too large' }, { status: 413 })
    const b64 = Buffer.from(buf).toString('base64')
    return NextResponse.json({ dataUrl: `data:${ct};base64,${b64}` })
  } finally { clearTimeout(timer) }
}
```

**Test:**

```
# Should work:
curl "http://localhost:3000/api/proxy-image?url=https://<storage-host>/img.png"
# Must fail (all of these):
curl "http://localhost:3000/api/proxy-image?url=http://169.254.169.254/"
curl "http://localhost:3000/api/proxy-image?url=http://localhost:54321/"
curl "http://localhost:3000/api/proxy-image?url=ftp://example.com/"
```

## 1.2 Lock down send-client-email

File: `src/app/api/send-client-email/route.ts` | Audit ref: C2

Two problems: no auth check and raw HTML interpolation. Both must be fixed together.

```
import { NextRequest, NextResponse } from 'next/server'
import { Resend } from 'resend'
import { requireAdminOrSuperAdmin } from '@/lib/server-admin-auth'

const resend = new Resend(process.env.RESEND_API_KEY)

function escapeHtml(s: string): string {
  return s.replace(/&/g,'&amp;').replace(/</g,'&lt;')
          .replace(/>/g,'&gt;').replace(/"/g,'&quot;').replace(/'/g,'&#39;')
}

export async function POST(req: NextRequest) {
  const auth = await requireAdminOrSuperAdmin(req)
  if (!auth.ok) return NextResponse.json({ error: auth.error }, { status: auth.status })

  const { email, name, status, password } = await req.json()
  if (!email || !name || !status)
    return NextResponse.json({ error: 'Missing required fields' }, { status: 400 })

  const safeName     = escapeHtml(String(name))
  const safeStatus   = escapeHtml(String(status))
  const safePassword = password ? escapeHtml(String(password)) : ''

  // Build subject + html from safeStatus, then send via resend...
  await resend.emails.send({
    from: 'noreply@yourdomain.com',  // <-- replace with verified Resend domain
    to: email, subject, html,
  })
  return NextResponse.json({ success: true })
}
```

After applying: verify a sending domain in Resend Dashboard > Domains, then replace `noreply@yourdomain.com`.

Also grep all callers to ensure they pass the auth token: `grep -rn "send-client-email" src/`

## 1.3 Replace invoice-token cryptography

File: `src/lib/invoice-token.ts` | Audit ref: C3

Drop crypto-js entirely. Replace with HMAC-SHA256 signed tokens with expiry.

> Breaking change: all existing tokens become invalid. Communicate this to the team and any known external recipients before shipping.

```
import crypto from 'node:crypto'

const SECRET = process.env.INVOICE_LINK_SECRET_KEY
if (!SECRET) throw new Error('INVOICE_LINK_SECRET_KEY env var is required')

const DEFAULT_TTL_SECONDS = 60 * 60 * 24 * 30  // 30 days
type Payload = { id: number; exp: number; purpose: 'view' | 'pay' }

// ... b64urlEncode / b64urlDecode helpers ...

function sign(payload: Payload): string {
  const body = b64urlEncode(Buffer.from(JSON.stringify(payload)))
  const mac  = crypto.createHmac('sha256', SECRET!).update(body).digest()
  return `${body}.${b64urlEncode(mac)}`
}

function verify(token: string): Payload | null {
  const [body, sig] = token.split('.')
  if (!body || !sig) return null
  const expected = b64urlEncode(crypto.createHmac('sha256', SECRET!).update(body).digest())
  if (!crypto.timingSafeEqual(Buffer.from(sig), Buffer.from(expected))) return null
  const p = JSON.parse(b64urlDecode(body).toString()) as Payload
  if (p.exp < Math.floor(Date.now() / 1000)) return null
  return p
}

export function encryptInvoiceId(id: number, purpose: 'view'|'pay' = 'view', ttl =
DEFAULT_TTL_SECONDS) {
  return sign({ id, exp: Math.floor(Date.now() / 1000) + ttl, purpose })
}
export function decryptInvoiceToken(token: string): number | null {
  return verify(token)?.id ?? null
}
```

Test: `decryptInvoiceToken(encryptInvoiceId(42))` should return 42; appending any character should return null.

### 1.4 Add server-side middleware

New file: `src/middleware.ts` | Audit ref: C4

```
import { type NextRequest, NextResponse } from 'next/server'
import { createServerClient } from '@supabase/ssr'

export async function middleware(request: NextRequest) {
  const response = NextResponse.next()
  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    { cookies: {
        getAll: () => request.cookies.getAll(),
        setAll: (c) => c.forEach(({ name, value, options }) =>
          response.cookies.set(name, value, options))
    } }
  )
  const { data: { user } } = await supabase.auth.getUser()
  if (request.nextUrl.pathname.startsWith('/dashboard') && !user) {
    const url = request.nextUrl.clone()
    url.pathname = '/login'
    url.searchParams.set('redirectTo', request.nextUrl.pathname)
    return NextResponse.redirect(url)
  }
  return response
```

```
}

export const config = {
  matcher: ['/dashboard/:path*'],
}
```

Test: Sign out. Open http://localhost:3000/dashboard/invoices in an incognito tab. Should redirect to /login?redirectTo=%2Fdashboard%2Finvoices.

### 1.5 Server-side the public invoice view

Files: `src/components/InvoiceView.tsx`, new `src/app/api/public/invoice/route.ts` | Context: prepares for RLS in Phase 3

Currently InvoiceView.tsx calls supabase from the client using the anon key. After RLS is on, anonymous users get 0 rows. Move the fetch server-side, gated behind token verification.

Create `src/app/api/public/invoice/route.ts`:

```
import { NextResponse } from 'next/server'
import { createClient } from '@supabase/supabase-js'
import { decryptInvoiceToken } from '@/lib/invoice-token'

const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY!
)

export async function GET(req: Request) {
  const token = new URL(req.url).searchParams.get('token')
  if (!token) return NextResponse.json({ error: 'Missing token' }, { status: 400 })

  const invoiceId = decryptInvoiceToken(token)
  if (!invoiceId) return NextResponse.json({ error: 'Invalid/expired token' }, { status: 401
})

  const { data: invoice } = await supabase
    .from('invoices')
    .select('*, employees!invoice_creator_id(employee_name), clients!client_id(name)')
    .eq('id', invoiceId).maybeSingle()

  const { data: brands } = await supabase.from('brands')
    .select('id, brand_name, brand_url, logo_url')
    .neq('isdeleted', true).order('brand_name')

  return NextResponse.json({ invoice, brands: brands ?? [] })
}
```

Then refactor `InvoiceView.tsx`: when a token is present, call `/api/public/invoice?token=...` instead of Supabase directly. Apply the same pattern to the payment page and any other anonymous invoice reads.

Test: Generate an invoice link from the dashboard. Open in incognito. Confirm it renders correctly.

# Phase 2 — API Route Auth Hardening

Estimated duration: 1 day

## 2.1 Gate payment-flow routes with invoice token

Files: `create-payment-intent/route.ts`, `stripe/config/route.ts`, `payment-gateways/validate-amount/route.ts`, `invoices/[id]/mark-paid/route.ts` | Audit ref: H1

All four routes use the service role but accept an unauthenticated invoiceId. Require a valid invoice token that binds to the requested invoice.

Pattern for POST routes:

```
import { decryptInvoiceToken } from '@/lib/invoice-token'

// inside POST handler:
const { invoiceId, token } = await req.json()
const tokenInvoiceId = token ? decryptInvoiceToken(String(token)) : null
if (!tokenInvoiceId || tokenInvoiceId !== Number(invoiceId)) {
  return NextResponse.json({ error: 'Invalid or missing invoice token' }, { status: 401 })
}
```

For GET routes, read token from query string: `const token = url.searchParams.get('token')`

Update frontend callers in `InvoicePayForm.tsx` and related components to include the token in every request.

## 2.2 Delete (or neuter) the mark-paid route

File: `src/app/api/invoices/[id]/mark-paid/route.ts` | Audit ref: H2

The Stripe webhook already marks invoices paid on `payment_intent.succeeded`. This route duplicates that logic without auth.

Preferred: delete it. Have the UI poll `/api/public/invoice` or listen to a Supabase Realtime subscription on invoices.status until the webhook updates the row.

If kept for UX: apply the token gate from 2.1 and remove all DB writes — return only a Stripe confirmation status, no database mutations.

## 2.3 Make the Stripe webhook idempotent

File: `src/app/api/webhooks/stripe/route.ts` | Audit ref: H3

- A — In create-payment-intent, write the payment_submissions row with the Stripe intent ID immediately.
- B — In the webhook, match by stripe_payment_intent_id, not by latest created_at:

```
const { data: paymentSubmission } = await supabase
  .from('payment_submissions')
  .select('id')
  .eq('invoice_id', Number(invoiceId))
  .eq('stripe_payment_intent_id', paymentIntent.id)  // <- add this
  .maybeSingle()
```

Also add a unique index to prevent duplicate rows on retries:

```
alter table public.payment_submissions
  add constraint payment_submissions_stripe_intent_uniq
  unique (stripe_payment_intent_id);
```

- C — Return 200 (not 500) for non-critical tracking-row update failures. Otherwise Stripe retry storms occur even though the invoice status has already been updated.

### 2.4 Fail at startup when env vars are missing

New file: `src/lib/env.ts` | Audit ref: H5

```
function required(name: string): string {
  const v = process.env[name]
  if (!v) throw new Error(`Missing required env var: ${name}`)
  return v
}

export const env = {
  SUPABASE_URL:              required('NEXT_PUBLIC_SUPABASE_URL'),
  SUPABASE_ANON_KEY:         required('NEXT_PUBLIC_SUPABASE_ANON_KEY'),
  SUPABASE_SERVICE_ROLE_KEY: required('SUPABASE_SERVICE_ROLE_KEY'),
  STRIPE_WEBHOOK_SECRET:     required('STRIPE_WEBHOOK_SECRET'),
  INVOICE_LINK_SECRET_KEY:   required('INVOICE_LINK_SECRET_KEY'),
  RESEND_API_KEY:            required('RESEND_API_KEY'),
  VAPID_PUBLIC_KEY:          required('VAPID_PUBLIC_KEY'),
  VAPID_PRIVATE_KEY:         required('VAPID_PRIVATE_KEY'),
}
```

Import `env` from this file in every server module that currently uses `process.env.X!`

# Phase 3 — RLS Rollout

Estimated duration: 1 day. Start Phase 3 only once Phases 0-2 are complete and merged.
Migration file: `supabase/migrations/20260417_enable_rls.sql`

## 3.1 Apply on a staging project first

- Create a staging Supabase project (or reuse an existing one).
- Restore a recent prod backup into staging.
- Point a staging deployment at it, then apply the migration:

```
supabase link --project-ref <staging-ref>
supabase db push
```

- Verify RLS is enabled on every table:

```
select schemaname, tablename, rowsecurity
from pg_tables where schemaname = 'public';
-- every row must show rowsecurity = true
```

- List all policies to confirm they exist:

```
select tablename, policyname, cmd, roles
from pg_policies where schemaname = 'public'
order by tablename, policyname;
```

## 3.2 Smoke-test as each role

| Role | Should Work | Should NOT Work |
|---|---|---|
| `superadmin` | Everything | — |
| `admin` | Create/edit/delete invoices, clients, brands, gateways | — |
| `user (employee/sales)` | Assigned clients' invoices + chat + own creations | Unassigned clients; payment_gateways rows |
| `client` | Own invoices, chat with handler, receive push | Other clients' invoices; payment_gateways |
| `anonymous` | Log in, register, view /invoice?token=..., pay via token | Load /dashboard/*, read any table via anon key |

Also verify anon-key reads are blocked:

```
# Should return [] or an error — not rows:
curl "https://<ref>.supabase.co/rest/v1/payment_gateways?select=*"
  -H "apikey: <anon-key>" -H "Authorization: Bearer <anon-key>"
```

## 3.3 Apply on prod

- Put the app into a maintenance banner if available.
- Take a fresh backup (see Phase 0.3).
- Apply the migration via Supabase dashboard or CLI.

- Run the pg_tables.rowsecurity check (same query as 3.1).
- Smoke-test each role against prod for ~15 minutes.
- Monitor error logs for PostgREST 401/403 spikes for the following hour.

### 3.4 Rollback plan

If a specific table is broken (fastest targeted fix):

```
alter table public.<tablename> disable row level security;
```

Full rollback across all affected tables:

```
do $$ declare t text;
begin
  foreach t in array array[
    'employees','clients','brands','invoices','payment_submissions',
    'payment_gateways','push_subscriptions','client_chat_messages',
    'client_chat_message_attachments'
  ] loop
    execute format('alter table public.%I disable row level security;', t);
  end loop;
end $$;
```

Policies can remain in place after rollback — they are inert without RLS enabled.

# Phase 4 — Schema Fixes

Estimated duration: 1 day. Phases 4-5 can proceed after the Supabase migration.

## 4.1 Migrate invoices.amount from TEXT to NUMERIC

Audit ref: M1. Run on staging first. Review spot-check output before committing.

```
begin;

alter table public.invoices add column amount_numeric numeric(12,2);

update public.invoices
set amount_numeric =
  nullif(regexp_replace(coalesce(amount, ''), '[^0-9.-]', '', 'g'), '')::numeric;

-- Spot-check before dropping:
select id, amount, amount_numeric from public.invoices order by id desc limit 20;

alter table public.invoices drop column amount;
alter table public.invoices rename column amount_numeric to amount;

commit;
```

Then update TypeScript types and remove defensive parseAmount() calls where no longer needed.

## 4.2 Change clients.isdeleted default to false

Audit ref: M2.

```
alter table public.clients alter column isdeleted set default false;
```

Audit all insert paths to confirm none relied on the previous default of true.

## 4.3 Backfill legacy chat attachments

Audit ref: H6.

```
insert into public.client_chat_message_attachments
  (message_id, attachment_name, attachment_path, sort_order, created_at)
select m.id, m.attachment_name, m.attachment_path, 0, m.created_at
from public.client_chat_messages m
where m.attachment_path is not null
  and not exists (
    select 1 from public.client_chat_message_attachments a
    where a.message_id = m.id
  );
```

- Update attachments/route.ts to stop writing to the legacy columns.
- Update all read paths to use client_chat_message_attachments only.
- After a soak period, drop the legacy columns:

```
alter table public.client_chat_messages drop column attachment_name;
alter table public.client_chat_messages drop column attachment_path;
```

## 4.4 Audit card data writes

Audit ref: H4.

```
grep -rn "card_last4\|card_expiry_month\|card_expiry_year\|name_on_card" src/
```

For every write found: confirm the source is a Stripe API response on the server, not a client form. If any client form is posting these fields, remove them and pull card details from `paymentIntent.latest_charge.payment_method_details.card` server-side instead.

## Phase 5 — Cleanup (Low findings)

No urgency. One PR per item, or bundle. Track as individual tickets.

| ID | Action |
|---|---|
| L2 | Delete dead gateway.secret_key branch in src/lib/server-stripe-gateways.ts (lines 124-133). |
| L3 | Pick one of avatar_path / avatar_url in employees table and drop the other. |
| L4 | Delete src/app/api/client-chat/[clientId]/attachments-v2/route.ts if no client references /attachments-v2. |
| L5 | Fix as unknown as cast in src/components/Invoice.tsx with proper typed interfaces. |
| L6 | Replace console.error with a structured JSON logger across all API routes. |
| L7 | Verify a real sending domain in Resend dashboard and update the from: field in send-client-email. |
| L8 | Replace key={i} with stable keys in chart cell components. |

# Final Checklist — Deploy Ready

Do not merge without a review. Do not apply to prod without staging first.

| Pre-deployment Checklist |
| --- |
| ☐ Phase 0 complete — Stripe keys rotated, DB backed up, all env vars validated |
| ☐ Phase 1 complete — all 5 critical fixes shipped and code-reviewed |
| ☐ Phase 2 complete — every unauthenticated payment API route is token-gated; mark-paid removed or read-only; webhook is idempotent |
| ☐ Phase 3 complete — RLS migration applied on staging, smoke-tested per role, then applied on prod |
| ☐ All Critical and High audit items resolved (reference AUDIT.md) |
| ☐ MEMORY.md updated if applicable |

**Reference documents:** AUDIT.md | supabase/migrations/20260417_enable_rls.sql | PROJECT_OVERVIEW.md

*End of Implementation Plan — Invoice-CRM Audit Remediation 2026-04-17*